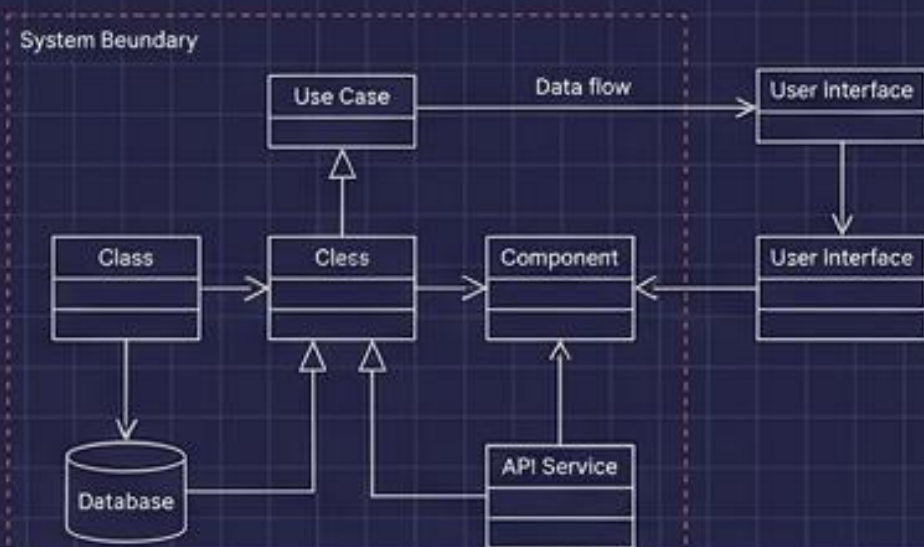




Ingeniería de Software.

Arquitectura – Diseño – Calidad

Resumen EA2

Bienvenidos a esta guía visual sobre los fundamentos de la Ingeniería de Software. Usaremos un ejemplo práctico que nos acompañará en todo el recorrido: la construcción de un Portal Inmobiliario. Veremos cómo una simple idea de negocio se transforma en diagramas (los planos), pasa por filtros de diseño (los materiales), se adapta al usuario (la fachada) y se sostiene sobre una arquitectura robusta (el rascacielos).

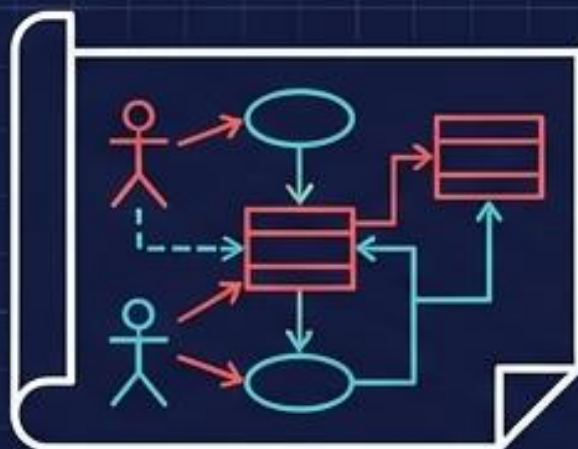
```
class RealEstatePortal {
  constructor(config) {
    this.db = new Database(config.db_url);
    this.api = new APIHandler();
  }
  async listProperties(filter) {
    const query = this.buildQuery(filter);
    const results = await this.db.execute(query);
    return this.api.formatResponse(results);
  }
}
```

```
interface Property {
  id: string;
  type: "house" | "apartment";
  price: number;
  location: GeoPoint;
  features: string[];
}
```

```
GET /api/properties?location=madrid&max_price=500000
=> [
  {
    id: "p1",
    type: "house",
    price: 450000,
    location: {
      lat: 40.4168,
      lon: -3.7838
    },
    features: ["3 beds", "garden"]
  },
  ...
]
```

¿Qué es UML y por qué lo necesitamos?

UML (Lenguaje Unificado de Modelado) no es un lenguaje de programación. Es una notación visual para modelar sistemas.



UML: El idioma universal

Cliente
(Negocio)

Traducción
(UML)

Desarrollador
(Código)



Evita confusiones técnicas entre equipos.



Transforma reglas de negocio en modelos claros.



Equivale a los planos arquitectónicos de un edificio.

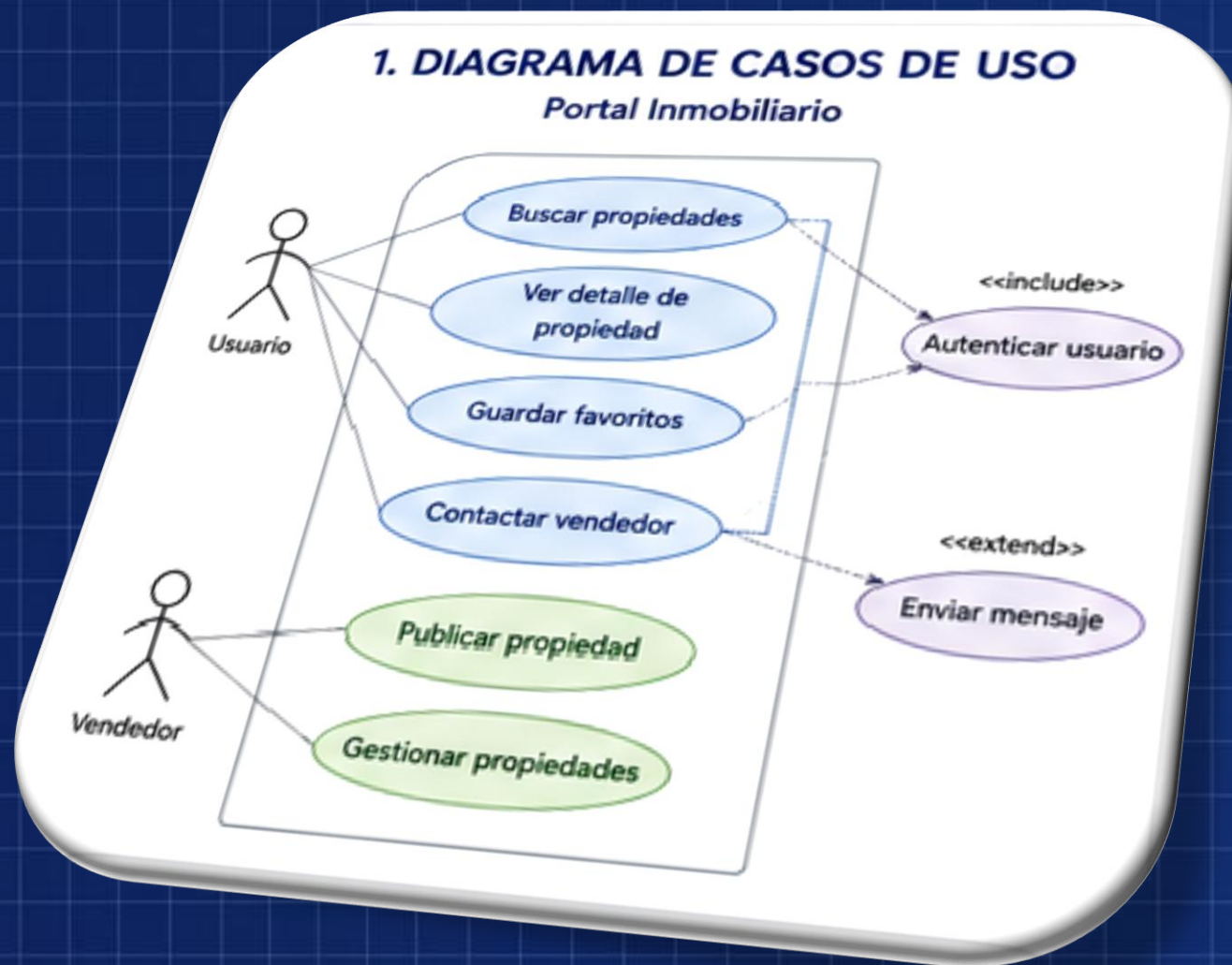
Si construyéramos un edificio solo conversando, el resultado sería un desastre. Necesitamos planos.

El cliente habla en términos de negocio, y el desarrollador habla en código.

UML es el puente visual entre ambos mundos, permitiendo que todos hablen el mismo idioma antes de escribir una sola línea de código.

El Comportamiento: Quién lo usa y Qué hace

Casos de Uso (El Qué)



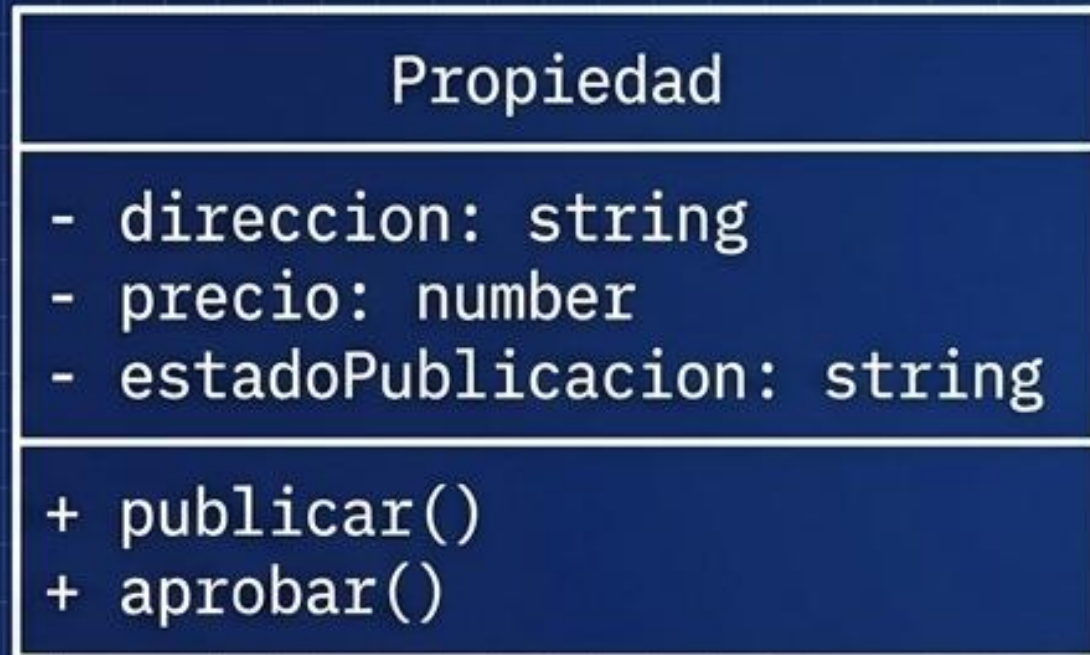
Actividades (El Paso a Paso)



Antes de programar, debemos entender el comportamiento del sistema. El Diagrama de Casos de Uso (izquierda) nos dice quién usa el sistema y qué puede hacer. A la derecha, el Diagrama de Actividades hace un zoom a ese caso de uso y muestra el flujo lógico y las decisiones (como validar datos) paso a paso.

La Estructura: Del Diagrama al Código

Estructura estática (Modelo UML)



Implementación real (Código)

```
class Propiedad {  
    private direccion: string;  
    private precio: number;  
    private estadoPublicacion: string;  
  
    public publicar() {  
        // lógica  
    }  
}
```

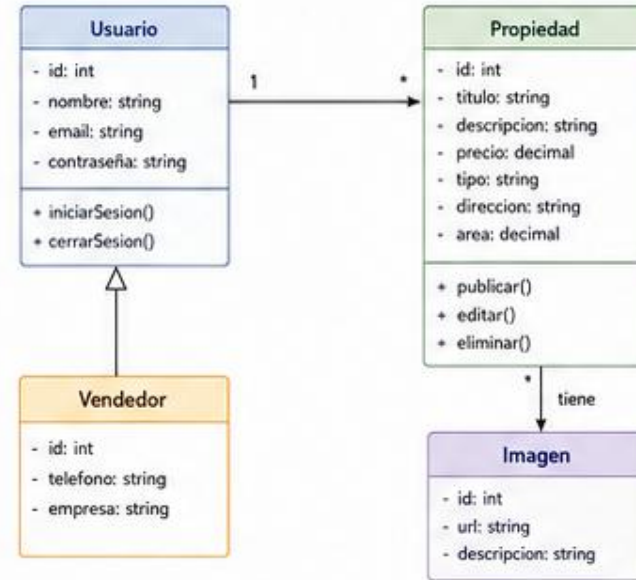
Aquí ocurre la magia. A la izquierda, vemos una caja UML que representa una 'Propiedad' con atributos (datos) y métodos (acciones). A la derecha, vemos cómo ese dibujo técnico se traduce línea por línea a código real. El símbolo '-' se convierte en 'private', y el '+' en 'public'. Así los planos se transforman en ladrillos digitales.

1. DIAGRAMA DE CASOS DE USO

Portal Inmobiliario

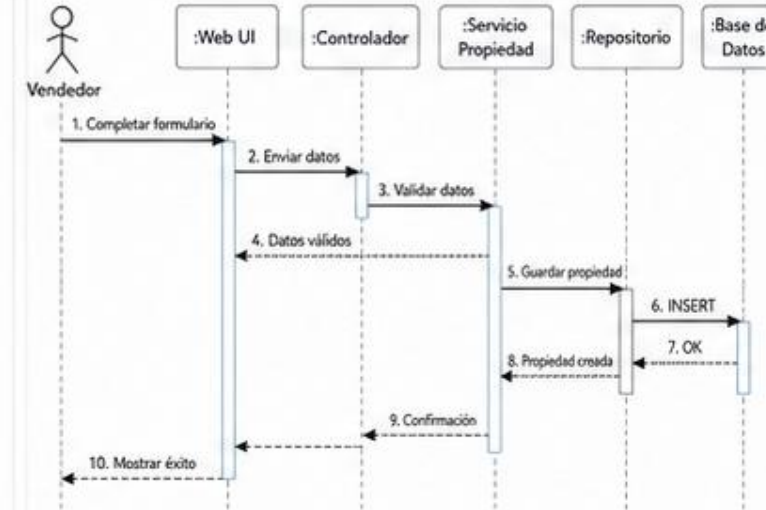


2. DIAGRAMA DE CLASES (SIMPLIFICADO)



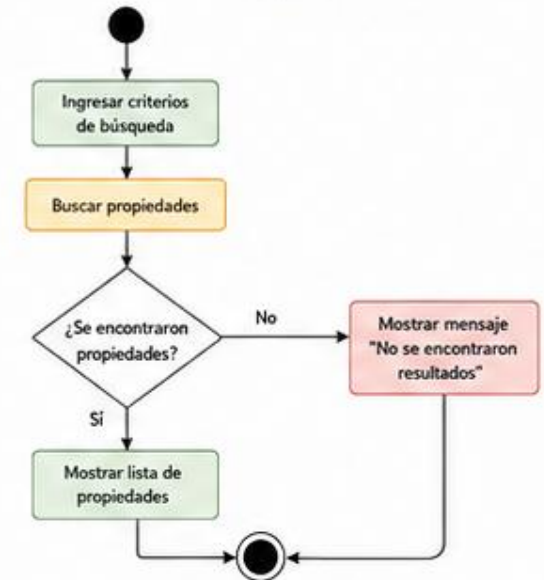
3. DIAGRAMA DE SECUENCIA

Publicar Propiedad



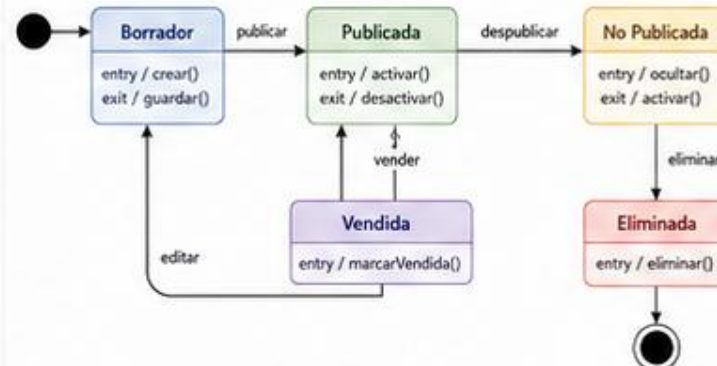
4. DIAGRAMA DE ACTIVIDADES

Buscar Propiedades



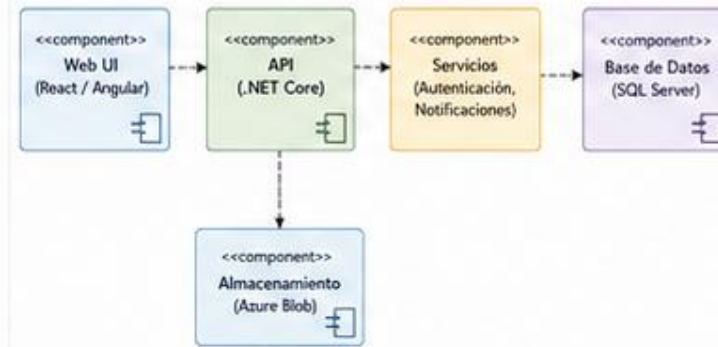
5. DIAGRAMA DE ESTADOS

Estado de una Propiedad

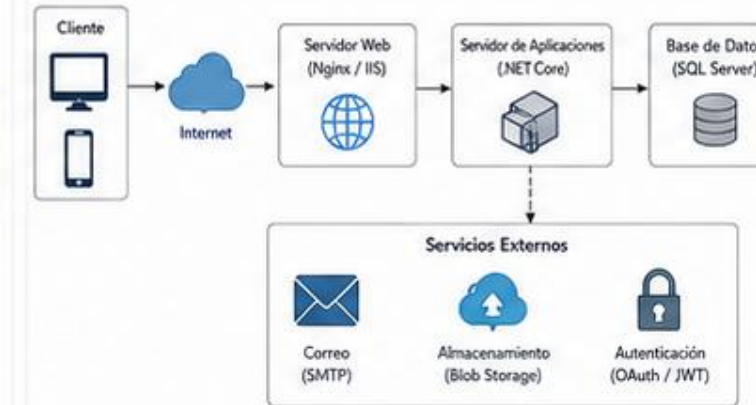


6. DIAGRAMA DE COMPONENTES

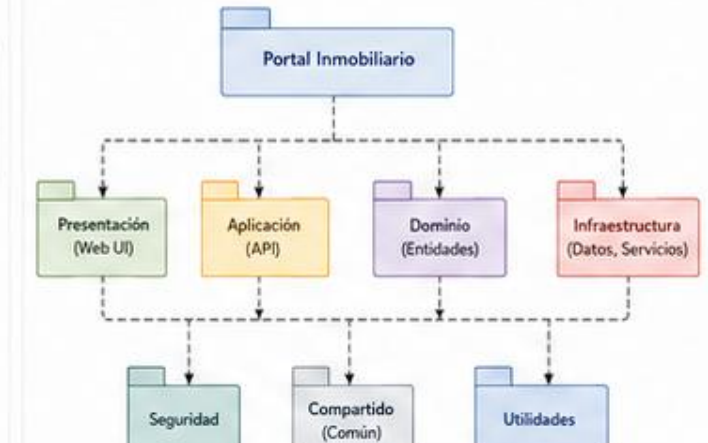
Portal Inmobiliario



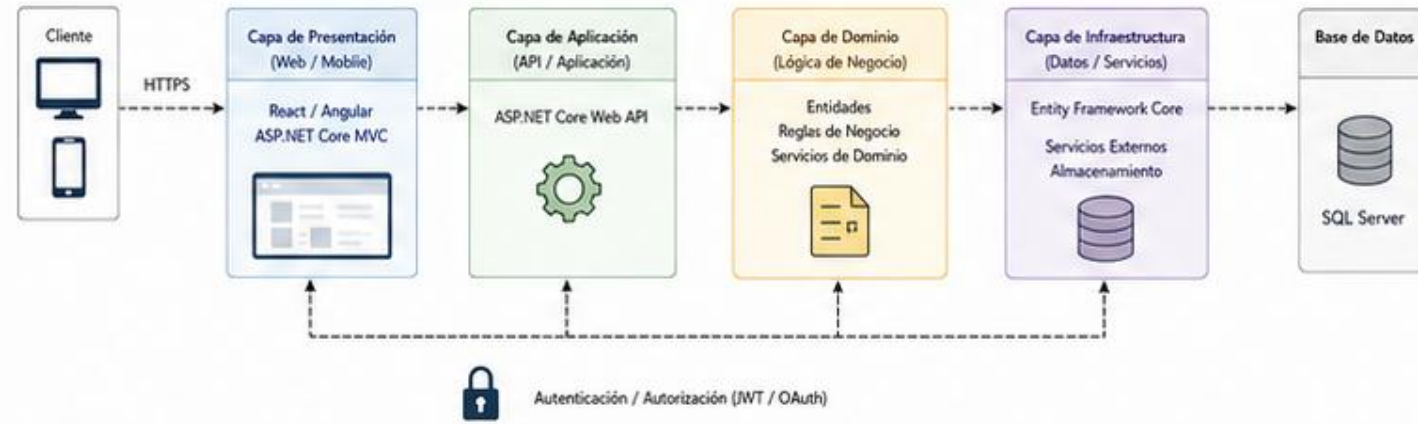
7. DIAGRAMA DE DESPLIEGUE



8. DIAGRAMA DE PAQUETES



9. DIAGRAMA DE ARQUITECTURA (VISTA LÓGICA EN CAPAS)



10. DIAGRAMA DE FLUJO CI/CD



Principios de Diseño: Abstracción y Encapsulación

Abstracción



Simplificar la realidad. Solo modelamos los datos que el portal necesita, ignorando el resto.

Encapsulación



Proteger los datos. Ocultamos el estado interno para que nadie lo altere sin validación.

Diseñar software es decidir qué incluir y qué esconder. Abstracción es extraer solo lo que le importa al portal (ignorando los hobbies del usuario). Encapsulación es un mecanismo de protección: escondemos la variable del estado en una bóveda, y solo permitimos modificarla usando métodos específicos ('llaves') que contienen reglas de validación.

La Regla de Oro: Alta Cohesión y Bajo Acoplamiento

Mala Práctica (PortalService Monolítico)



El Problema (Baja Cohesión / Alto Acoplamiento):

Si todo está mezclado, cambiar el sistema de pagos rompe la búsqueda de propiedades.

Buena Práctica (Arquitectura Modular)



La Solución (Alta Cohesión / Bajo Acoplamiento):

Módulos independientes con responsabilidades únicas y enfocadas.

La Cohesión mide qué tan enfocada está una parte del sistema (hacer una sola cosa bien). El Acoplamiento mide qué tanto dependen los módulos entre sí. Queremos módulos altamente cohesivos y con bajo acoplamiento: si el día de mañana cambiamos la pasarela de pagos, el módulo de búsqueda no debería enterarse ni romperse.

Calidad de Software: Más allá del "Funciona"

"Un software puede funcionar hoy y aun así estar mal diseñado."



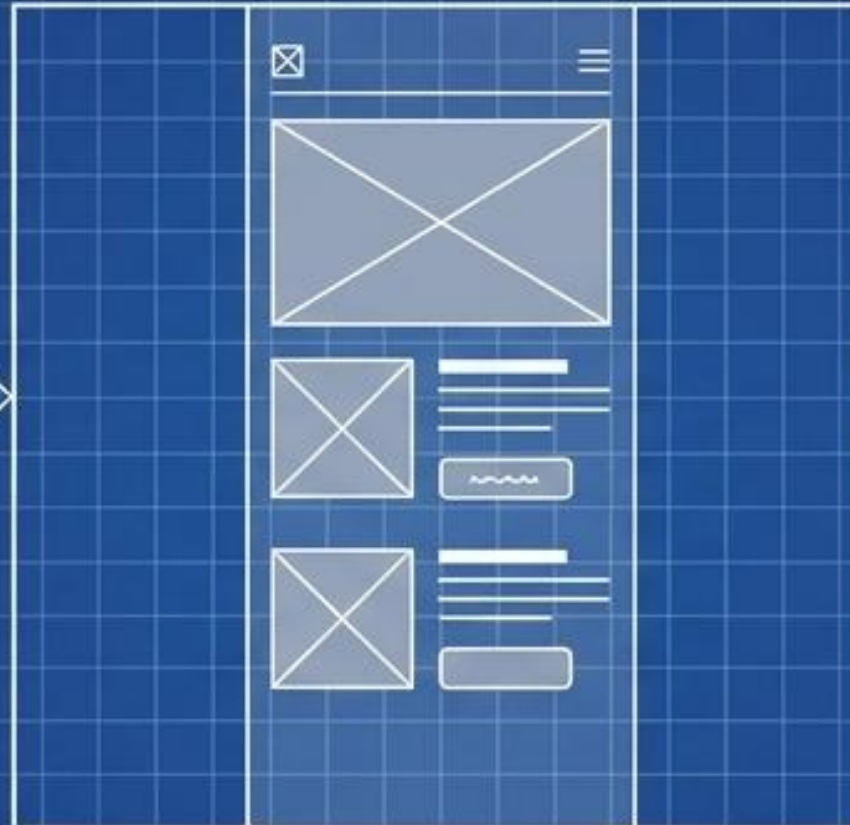
Es un error común pensar que si el código compila, el software es de calidad. La calidad es multidimensional. Un portal puede ser muy rápido (rendimiento), pero si cambiar una pequeña regla de negocio causa errores en cadena por todo el sistema, **falla en mantenibilidad**. El diseño asegura que el software sobreviva a largo plazo.

Diseño de Interfaz (UI) y Prototipado

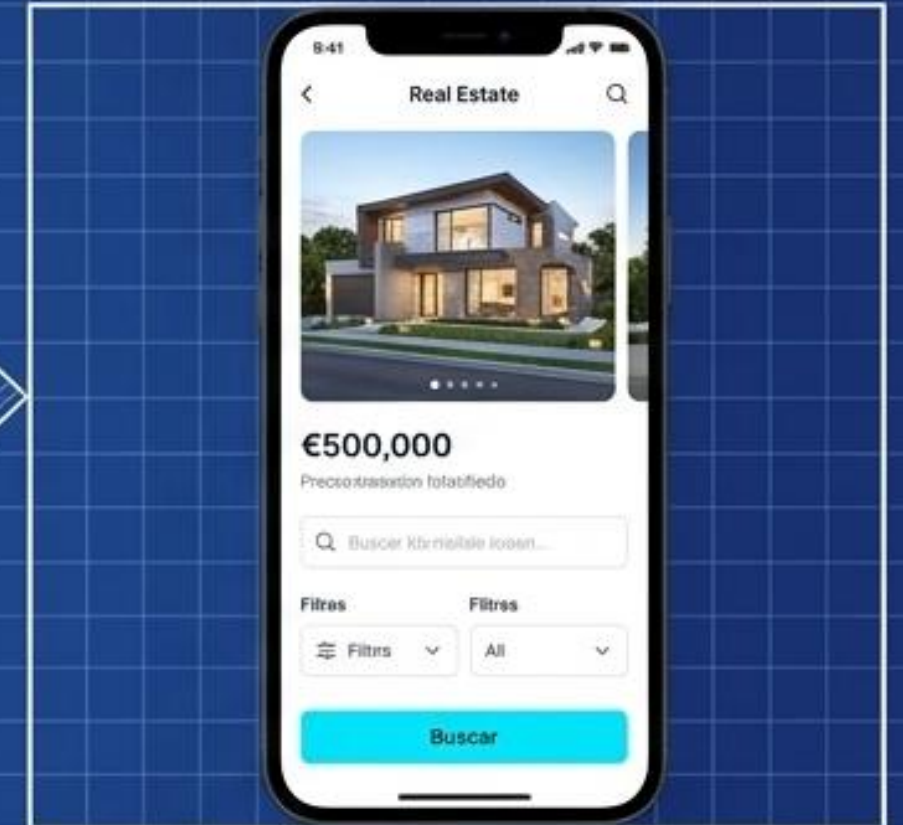
Baja Fidelidad (Boceto)



Media Fidelidad (Wireframe)



Alta Fidelidad (Mockup)



El prototipado valida requisitos y navegación antes de invertir costosas horas de programación.

Diseñar la interfaz no es abrir Photoshop de inmediato; es un proceso de validación. Comenzamos con dibujos en papel para validar la idea. Luego pasamos a wireframes grises, y finalmente a prototipos navegables. Equivocarse en un dibujo toma 5 minutos; equivocarse y reescribir código en producción toma semanas.

Usabilidad en la Práctica (Heurísticas de Nielsen)

Un sistema usable es claro, eficiente y tolerante a los errores humanos.

The mockup shows a form titled 'Publicar Propiedad' with the following fields: 'Título', 'Descripción', 'Precio', 'Tipo de Propiedad' (a dropdown menu currently showing 'All'), and 'Ubicación'. Below these is a 'Fotos' section with a gallery containing one image of a house and a placeholder with a loading spinner and the text 'Cargando imágenes...'. At the bottom right is a 'Publicar' button. A red circle with the number '3' is placed next to the 'Título' field. A red circle with the number '2' is placed over the 'Publicar' button, which has a tooltip above it that says 'Falta ingresar precio'. A red circle with the number '1' is placed over the 'Cargando imágenes...' placeholder.

1

Visibilidad del estado:
"Cargando imágenes..."

2

Prevención de errores: Botón deshabilitado si faltan datos.

3

Diseño estético y minimalista:
Mostrar solo los campos esenciales.

Jakob Nielsen definió heurísticas para sistemas fáciles de usar. Aquí vemos tres. 1) Visibilidad: el usuario nunca debe preguntarse qué hace el sistema; mostramos un "cargando". 2) Prevención: en lugar de dejar que envíe un formulario vacío y darle un error, deshabilitamos el botón de publicar. 3) Minimalismo: sin botones inútiles.

Accesibilidad (Principios WCAG): Software para Todos

1. Perceptible

`<img alt="Casa de 3 pisos en Ñuñoa">`



2. Operable

Buscar...

Tab



3. Comprensible



Error 500



Ingrese un correo
válido con formato
@dominio.com

4. Robusto



Un sistema estético pero inaccesible tiene un diseño deficiente.

La accesibilidad es un estándar técnico. Los principios WCAG garantizan que cualquier persona use el portal.

Perceptible: las imágenes deben tener texto alternativo. Operable: debe poder navegarse usando solo teclado.

Comprensible: los errores deben estar en lenguaje humano. Robusto: el código debe funcionar en diferentes navegadores.

¿Qué es la Arquitectura de Software?

La arquitectura define la estructura fundamental, los componentes principales y cómo se comunican.

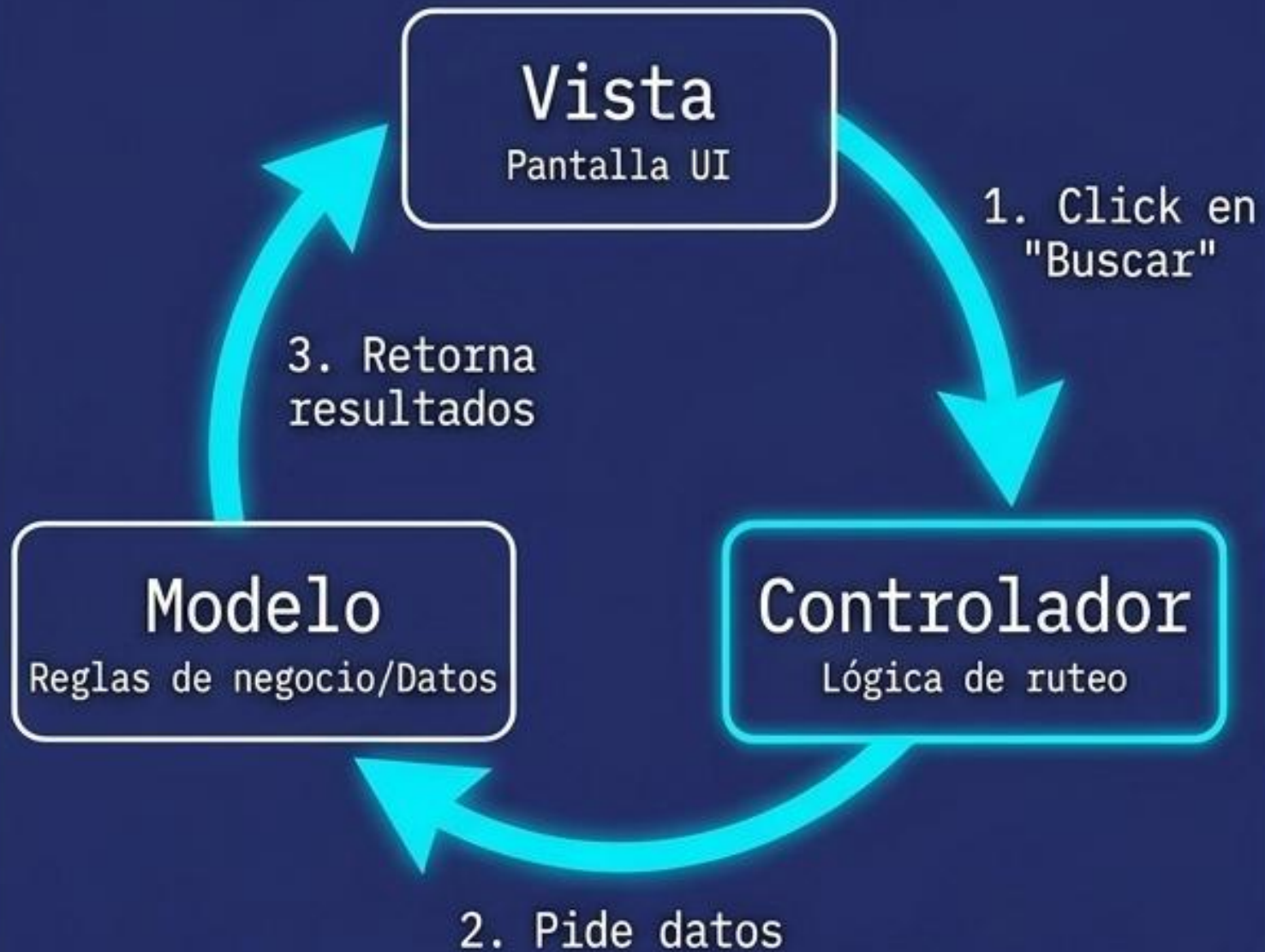


Diseño (Micro)	Arquitectura (Macro)
¿Cómo funciona la clase Propiedad interna?	¿Dónde vive esa clase y cómo el servidor web consulta la base de datos?

Si el diseño UML nos dice cómo se distribuyen las habitaciones de un departamento, la Arquitectura nos dice cómo están estructurados los cimientos del edificio completo y cómo se conecta a la red eléctrica de la ciudad. Es el nivel más alto de decisiones técnicas.

Patrones Fundamentales: Modelo-Vista-Controlador (MVC)

El patrón organiza el flujo.



La arquitectura dicta la organización real del código.



En arquitectura, usamos patrones que son soluciones comprobadas. MVC es un clásico. Imagina un restaurante: la Vista es el menú, el Controlador es el mesero que toma el pedido, y el Modelo es la cocina. El mesero nunca cocina. Lo fascinante es que este concepto se traduce literalmente en cómo organizamos las carpetas en el código.

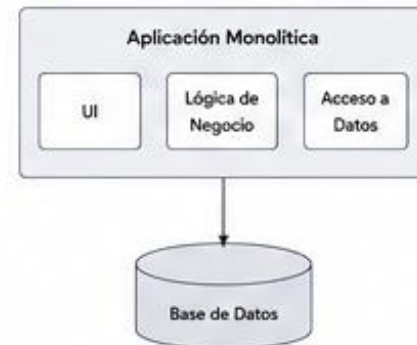
PATRONES DE ARQUITECTURA DE SOFTWARE



1. MONOLÍTICO

Toda la funcionalidad de la aplicación está integrada en una única unidad desplegable.

Diagrama



Ventajas

- Fácil de desarrollar al inicio
- Despliegue simple
- Buen rendimiento en aplicaciones pequeñas

Casos de uso

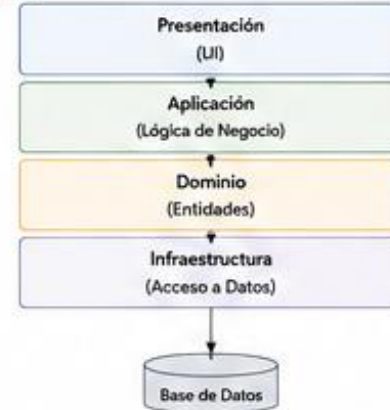
- Aplicaciones pequeñas
- Equipos pequeños
- MVPs



2. ARQUITECTURA EN CAPAS (LAYERED)

Organiza la aplicación en capas lógicas con responsabilidades bien definidas.

Diagrama



Ventajas

- Separación de responsabilidades
- Mantenible y testeable
- Fácil de evolucionar

Casos de uso

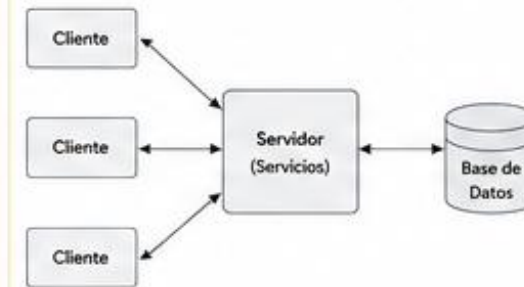
- Aplicaciones empresariales
- Sistemas medianos y grandes



3. ARQUITECTURA CLIENTE-SERVIDOR

Divide el sistema en clientes que solicitan servicios y servidores que los proveen.

Diagrama



Ventajas

- Escalabilidad
- Centralización de datos y lógica
- Fácil mantenimiento

Casos de uso

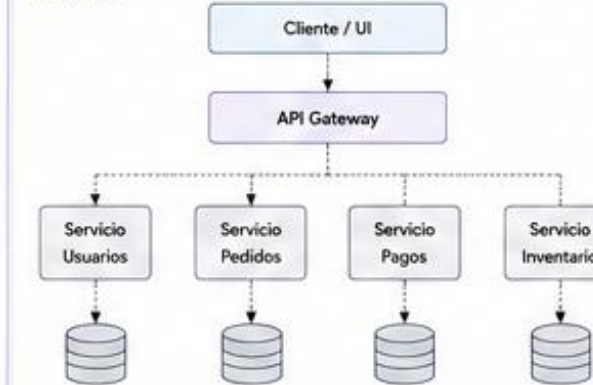
- Aplicaciones distribuidas
- Sistemas de red
- Aplicaciones web tradicionales



4. ARQUITECTURA MICROSERVICIOS

La aplicación se compone de servicios pequeños, independientes y desplegables de forma autónoma.

Diagrama



Ventajas

- Alta escalabilidad
- Despliegue independiente
- Tecnología heterogénea

Casos de uso

- Sistemas grandes y complejos
- Equipos distribuidos
- Aplicaciones en la nube



5. ARQUITECTURA ORIENTADA A EVENTOS (EVENT-DRIVEN)

Los componentes se comunican mediante eventos asíncronos.

Diagrama



Ventajas

- Desacoplamiento
- Alta escalabilidad
- Respuesta en tiempo real

Casos de uso

- Sistemas en tiempo real
- Integración de sistemas
- IoT, notificaciones, streaming



6. ARQUITECTURA PIPELINE Y FILTER (TUBERÍAS Y FILTROS)

Los datos pasan por una serie de filtros (etapas) que los transforman.

Diagrama



Ventajas

- Reutilizable
- Mantenible
- Fácil de extender

Casos de uso

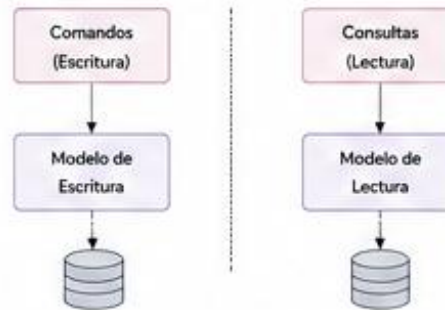
- Procesamiento de datos
- Compiladores
- ETL (Extract, Transform, Load)



7. ARQUITECTURA CQRS (COMMAND QUERY RESPONSIBILITY SEGREGATION)

Separa las operaciones de lectura (Query) de las de escritura (Command).

Diagrama



Ventajas

- Optimiza lecturas y escrituras
- Escalabilidad
- Flexibilidad en los modelos

Casos de uso

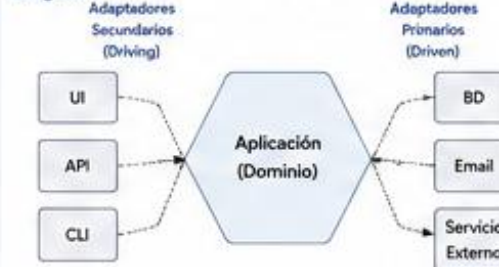
- Sistemas con muchas lecturas
- Reporting y analítica
- Aplicaciones complejas



8. ARQUITECTURA HEXAGONAL (PUERTOS Y ADAPTADORES)

Aísla la lógica de negocio del mundo externo usando puertos y adaptadores.

Diagrama



Ventajas

- Desacoplado
- Testeable
- Independiente de tecnología

Casos de uso

- Aplicaciones complejas
- Requerimientos cambiantes
- Test automatizado



9. ARQUITECTURA CLEAN

Basada en Hexagonal, enfatiza en la separación de capas y reglas de dependencias.

Diagrama



Ventajas

- Independiente de frameworks
- Alta mantenibilidad
- Reglas claras de dependencias

Casos de uso

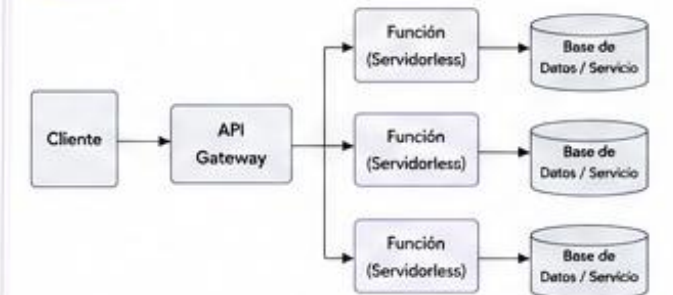
- Proyectos a largo plazo
- Equipos grandes
- Código fácilmente testeable



10. ARQUITECTURA SERVERLESS

La infraestructura es gestionada por el proveedor y el código se ejecuta bajo demanda.

Diagrama



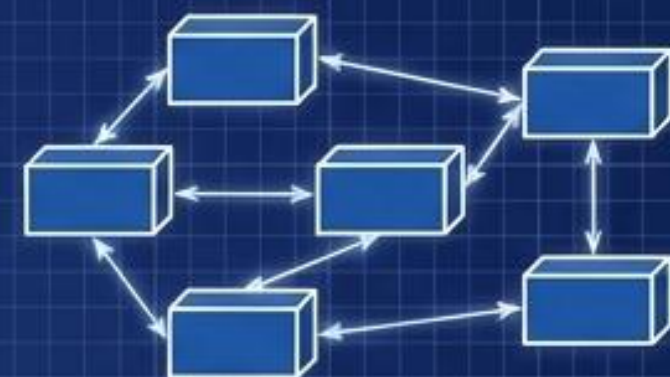
Ventajas

- Escalabilidad automática
- Sin gestión de servidores
- Pago por uso

Casos de uso

- Aplicaciones event-driven
- APIs y backends móviles
- Procesamiento por lotes

Evolución del Sistema: Monolito vs. Microservicios

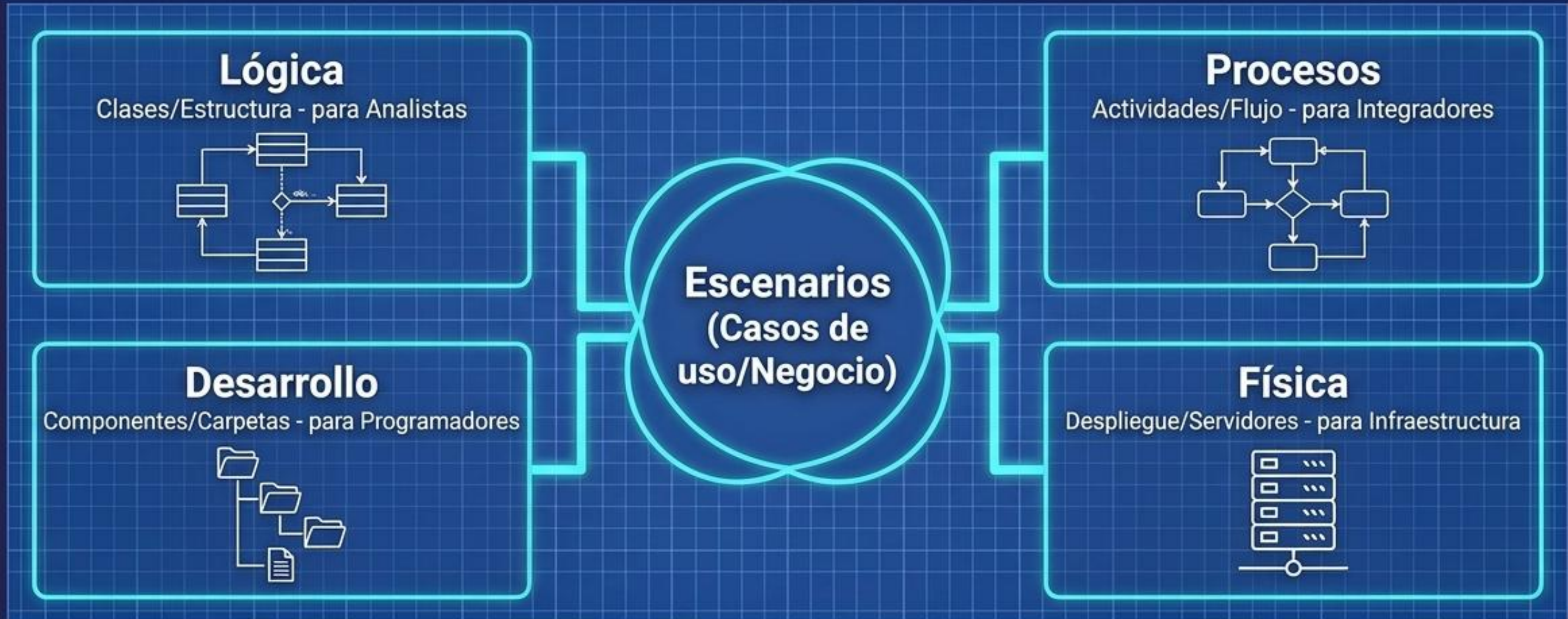


Dimensión	Monolito Modular	Microservicios
Estructura	Todo en una sola aplicación	Servicios independientes
Complejidad	Baja (Fácil despliegue inicial)	Alta (Requiere DevOps)
Escalabilidad	Escala todo el bloque	Escala servicios por separado
Contexto Ideal	Startups, validar negocio rápido	Sistemas masivos, alto tráfico

¿Cómo debería nacer nuestro portal? Al principio, un Monolito Modular es perfecto para validar el negocio rápido. Pero si el portal es un éxito y las 'Búsquedas' consumen todos los recursos mientras que los 'Pagos' casi no tienen tráfico, evolucionamos a Microservicios, separando la búsqueda en un servidor independiente que puede escalar por sí solo.

El Modelo 4+1: Las 5 Vistas de la Arquitectura

Un mismo sistema requiere diferentes lentes dependiendo de quién lo observe.



Nota Un sistema de software es tan complejo que no se describe con un solo diagrama. El ingeniero de infraestructura necesita la Vista Física (dónde están los servidores). El programador mira la Vista de Desarrollo (los componentes). El analista la Vista Lógica. Y todas convergen en el centro: la Vista de Escenarios, los requerimientos del negocio. Todas son verdaderas, desde distintos ángulos.

Conclusión: El Camino del Software

Idea de
Negocio

Modelado
(UML)

Diseño
(Calidad)

UI/UX

Arquitectura

**La mejor respuesta en ingeniería:
Depende del contexto.**

No existe un patrón universalmente superior. La decisión correcta justifica la tecnología basándose en las necesidades del usuario, el tamaño del equipo y la escala del negocio.

Nota Hemos recorrido el camino desde traducir una idea con UML, diseñar con calidad, adaptar la fachada con UI/UX, hasta estructurar los cimientos. Si se llevan una lección hoy, que sea esta: nunca digan que una tecnología es inherentemente "la mejor". Todo depende del contexto. La excelencia técnica es evaluar ese contexto y diseñar la solución exacta que el problema necesita.